Chủ đề #5:

ĐỆ QUY & QUAY LUI

Trong đời sống hằng ngày, chúng ta đã gặp rất nhiều vật, hiện tượng mà bản thân nó là vật chứa cho chính nó như: sách giáo khoa Toán lớp 3 (chương trình cũ), cửa sổ OBS, google meet khi quay màn hình, ... Trong lập trình, những bài toán có tính chất tương tự đã sinh ra khái niệm "đệ quy".

I. ĐỆ QUY *(RECURSION)*:

1. Khái niệm:

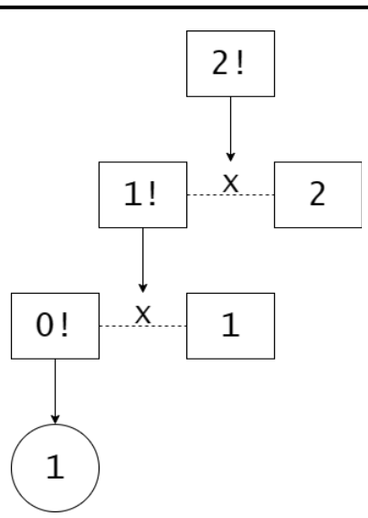
Ta gọi một đối tượng là đệ quy nếu nó được định nghĩa qua chính nó hoặc một đối tượng cùng dạng qua quy nạp. Trong lập trình, người ta thường cài đệ quy là một hàm (hoặc thủ tục) tự gọi chính nó.

2. Ví dụ với phép giai thừa:

Ta có ***n! = (n - 1)! × n***. Như vậy, để tính n! thì ta phải tính ***(n - 1)!*** và tương tự, phải tính ***(n - 2)!*** ; ***(n - i)!*** ; ... Tuy nhiên, nếu cứ thực hiện phép lặp trên thì "liệu bài toán sẽ đi về đâu?". Để trả lời cho câu hỏi ấy, khái niệm "trường hợp cơ sở" đã ra đời với sứ mệnh xác định điểm dừng cho đệ quy.

Xét thấy $n! = (n-1)! \times n$ $\Leftrightarrow (n-1)! = \frac{n!}{n}$ $\Rightarrow \begin{cases} 1! = \frac{2!}{2} = 1 \\ 0! = \frac{1!}{1} = 1 \end{cases}$	• Với bài toán tính giai thừa, ta đã xác định được trường hợp cơ sở là n = 0 hoặc n = 1. • Khi biểu thức đạt đến trường hợp cơ sở, ta cần nhận kết quả và thoát đệ quy.

Cây đệ quy *(với n = 2)*	**Cài đặt - O(n)**

	`int GiaiThua(int n)`
	`{`
	`  if (n == 1 || n == 0) return 1;`
	`  return GiaiThua(n - 1) * n;`
	`}`
	
	`/* n = 0 hoặc n = 1 là`
	`  trường hợp cơ sở`
	`  ------`
	`  n! = (n - 1)! * n là`
	`  công thức truy hồi  */`

→ Trong các bài toán dùng đệ quy, trường hợp cơ sở cần được xác định đúng và rõ ràng. Vì khi không có trường hợp cơ sở cụ thể sẽ dẫn đến việc đệ quy vô hạn. Trong ví dụ trên, nếu không có trường hợp cho rằng 0! = 1 thì chương trình sẽ tiếp tục đệ quy và tính 0! = (-1)! * 0 (vô lý). Khi đệ quy quá nhiều lần sẽ dẫn đến tràn bộ nhớ và chạy sinh lỗi

3. Ứng dụng trong dãy Fibonacci:

Dãy Fibonacci là một dãy số được định nghĩa bằng công thức truy hồi sau: *F(n) = F(n - 1) + F(n - 2)* với *F(0) = 0* và *F(1) = 1*. Nói cách khác, trong dãy Fibonacci, mỗi phần tử là tổng của hai phần tử liền trước nó. *(Trong một số tài liệu, người ta quy định F(0), F(1) khác một chút nhưng về bản chất thì không thay đổi)*.

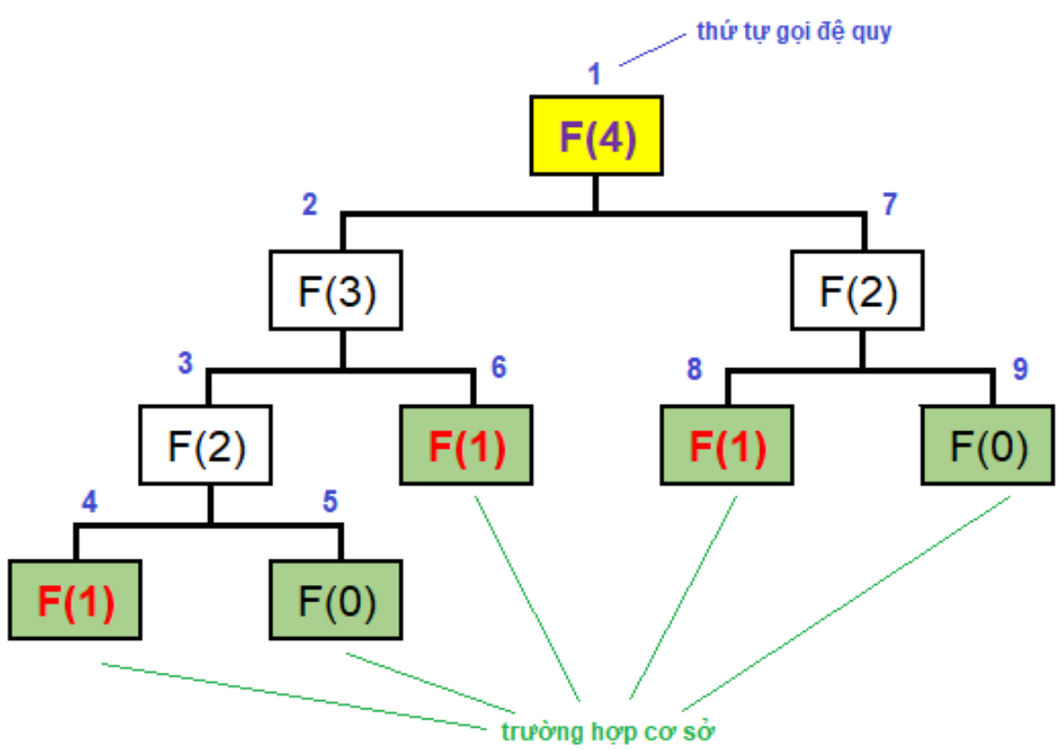
Bài toán: Nhập vào ***n*** *(n ⩽ 20). Xác định số Fibonacci thứ* ***n***. Theo định nghĩa của dãy Fibonacci, ta đã xác định được:

- Trường hợp cơ sở: F(0) = 0; F(1) = 1.
- Công thức truy hồi: F(n) = F(n - 1) + F(n - 2).

Cài đặt - O(2^n):

```
int Fibo(int n) {
    if (n == 0) return 0;     // trường hợp cơ sở
    if (n == 1) return 1;     // trường hợp cơ sở
    return Fibo(n - 1) + Fibo(n - 2);     // đệ quy (CT truy hồi)
}
```

Hãy cùng quan sát và phân tích cây đệ quy với n = 4:



- Cài đặt ngắn gọn, dễ hiểu, dễ phân tích.
- Chỉ với ***n*** = ***4*** mà chương trình phải gọi hàm đệ quy đến ***9*** lần.
- Hàm tính ***F(0)***, ***F(1)*** và ***F(2)*** đều được gọi trên 1 lần. Đặc biệt, hàm tính ***F(1)*** còn được gọi đi gọi lại đến ***3*** lần.

→ Thông thường, nếu không sử dụng các kỹ thuật tối ưu cho đệ quy, phương pháp này thậm chí sẽ có độ phức tạp lớn hơn cả vét cạn. Tuy nhiên, tùy vào tính chất của bài toán, trong một số trường hợp, sử dụng đệ quy sẽ mang lại hiệu quả tốt hơn các phương pháp cơ bản khác. Để khắc phục nhược điểm này của đệ quy, bọn mình sẽ có tài liệu về Quy hoạch động trong chủ đề tháng 4/2024.

Ghi nhớ: Nếu một bài toán ***T*** có lời giải được thực hiện bằng một bài toán con ***T'*** có dạng giống ***T*** thì đó là một giải thuật đệ quy. ***T'*** phải là một bài toán đơn giản hơn ***T*** *(dữ liệu nhỏ hơn, độ phức tạp nhỏ hơn,...)* và không cần đến ***T*** để giải nó.

Một bài toán đệ quy có lời giải gồm 2 phần:

- **Trường hợp cơ sở:** là phần có thể giải trực tiếp mà không cần một bài toán con nào. Đây là điểm dừng của đệ quy.
- **Đệ quy:** gọi ra bài toán con để giải bài toán hiện tại. Phần này cần thực hiện đến khi bài toán được đưa về trường hợp cơ sở.

Chú ý khi dùng đệ quy:

- Phân tích kỹ, xác định đúng tính chất bài toán trước khi quyết định giải bằng đệ quy.
- Không lạm dụng đệ quy khi không cần thiết.
- Ở một số bài không phù hợp mà vẫn cố dùng đệ quy sẽ dẫn đến tràn stack.

Các bước cài đặt đệ quy:

- **B1:** Xác định đúng trường hợp cơ sở (nếu xác định sai, chương trình sẽ lặp vô hạn hoặc trả về kết quả sai).
- **B2:** Xác định công thức truy hồi (tùy vào yêu cầu và tính chất bài toán).
- **B3:** Cài đặt (nên vẽ nháp cây đệ quy khi chưa quen).

II. QUAY LUI *(BACKTRACKING)*:

1. Khái niệm:

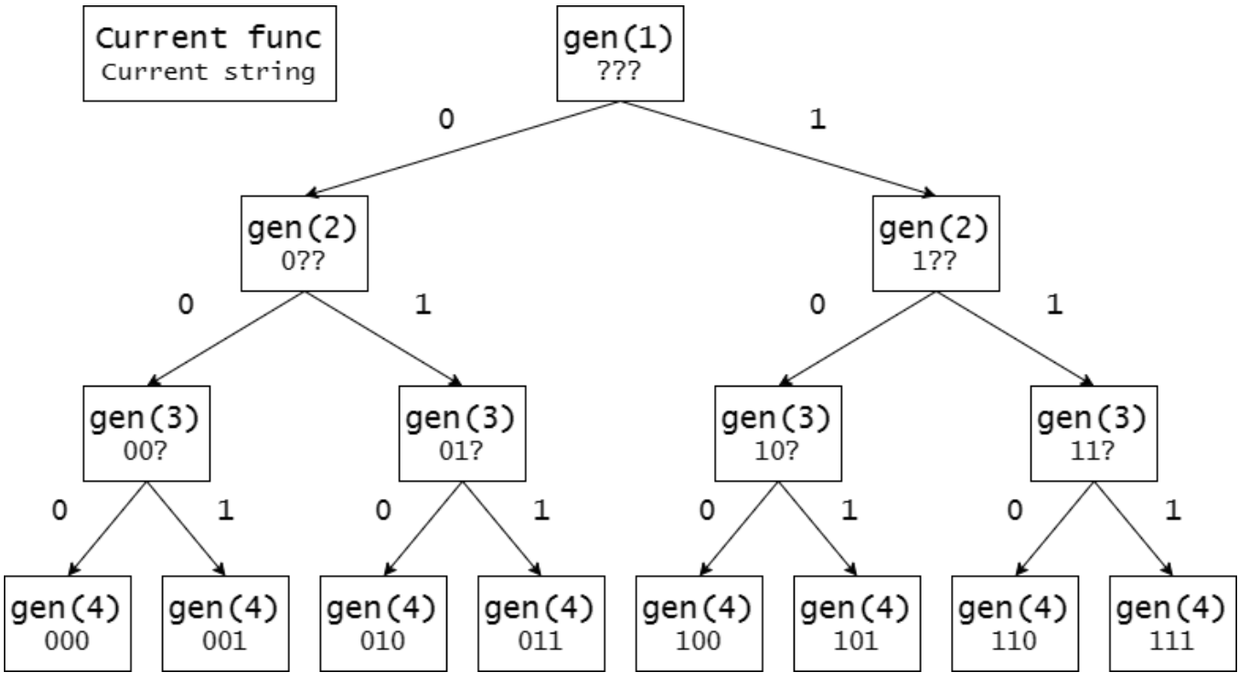
Thuật toán quay lui được sử dụng trong các bài toán liệt kê các cấu hình mà mỗi cấu hình được sinh bằng cách xây dựng từng phần tử, mỗi phần tử được chọn từ tất cả các cách chọn. Đây cũng có thể xem là vét cạn.

Ví dụ: liệt kê tất cả các dãy nhị phân có độ dài là 3.

- Xét vị trí đầu tiên: ta có thể chọn 0 hoặc 1, bắt đầu chọn từ 0.
- Với mỗi trường hợp của ký tự thứ nhất, xét tiếp mọi trường hợp ký tự thứ hai. Ta được 00, 01 từ 0 ở bước trước và 10, 11 từ 1 ở bước trước.
- Tương tự, với mỗi trường hợp của ký tự thứ hai, ta xét mọi trường hợp ở ký tự thứ ba. Các dãy nhận được là 000, 001, 010, 011, 100, 101, 110, 111.

→ Sinh được 8 dãy thỏa yêu cầu: *000, 001, 010, 011, 100, 101, 110, 111*

Sơ đồ quay lui sinh dãy nhị phân



Cài đặt - O(2^n):

```
int n; string s = "";
void generate(int pos) {
    if (pos > n) {          // Trường hợp cơ sở
        cout << s << '\n';
        return;
    }
    for (char i = '0'; i <= '1'; i ++) {
        s.push_back(i);     // thêm ký tự mới vào dãy
        generate(pos + 1); // đệ quy sinh kí tự ở vị trí kế tiếp
        s.pop_back();       // bỏ ký tự này đi
    }
}
int main() {
    cin >> n;
    generate(1);
    return 0;
}
```

2. Nói thêm về Tổ hợp, Chỉnh hợp, Hoán vị:

	Định nghĩa	Công thức
Hoán vị	Cho tập hợp A gồm n phần tử Mỗi kết quả của sự sắp xếp thứ tự của n phần tử của tập hợp A được gọi là một hoán vị của n phần tử đó	$\boldsymbol{P_n}$: số hoán vị của n phần tử $\boldsymbol{P_n = n \times (n-1) \times \cdots \times 2 \times 1 = n!}$
Chỉnh hợp	Cho tập hợp A gồm n phần tử $(n \geq 1)$. Kết quả của việc lấy k phần tử khác nhau từ n phần tử của tập hợp A và sắp xếp chúng theo một thứu tự nào đó được gọi là chỉnh hợp chập k của n phần tử.	$\boldsymbol{A_n^k}$: số các chỉnh hợp chập k của n phần tử $(1 \leq k \leq n)$. $\boldsymbol{A_n^k = n \times (n-1) \times \cdots \times (n-k+1)}$ $\boldsymbol{A_n^k = \frac{n!}{(n-k)!}}$
Tổ hợp	Giả sử tập A có n phần tử $(n \geq 1)$. Mỗi tập con gồm k phần tử của A được gọi là một phần tử chập k của n phần tử đã cho.	$\boldsymbol{C_n^k}$: số các tổ hợp chập k của n phần tử $(0 \leq k \leq n)$ $\boldsymbol{C_n^k = \frac{n!}{k! \times (n-k)!}}$ ***Tính chất:*** - $(0 \leq k \leq n)$: $\boldsymbol{C_n^k = C_n^{n-k}}$ - $(1 \leq k \leq n)$: $\boldsymbol{C_n^k = C_{n-1}^{k-1} + C_{n-1}^k}$

Bài tập củng cố:

Hoán vị:

Có thể lập bao nhiêu số có 6 chữ số khác nhau không bắt đầu bởi 34 được lập từ các số 1, 2, 3, 4, 5, 6?

<u>Lời giải:</u>

- Số các số có 6 chữ số khác nhau được lập từ 1, 2, 3, 4, 5, 6 là 6! = 720 số
- Số các số có 6 chữ số khác nhau được lập từ 1, 2, 3, 4, 5, 6 mà bắt đầu bằng 34 là 4! = 24 số
- Số các số có 6 chữ số khác nhau không bắt đầu bởi 34 được lập từ 1, 2, 3, 4, 5, 6 là 720 – 24 = 695 số

Chỉnh hợp:

Từ các số 0, 1, 3, 6, 9 có thể lập được bao nhiêu số chẵn gồm 4 chữ số khác nhau.

Lời giải:

- Gọi số có 4 chữ số khác nhau là $\overline{abcd}$ $(a \neq b \neq c \neq d)$. Vì số cần tìm là số chẵn nên d có 2 cách chọn là 0 hoặc 6.
    - Nếu d = 0 thì có 1 cách chọn d.
      Khi đó có $\boldsymbol{A_4^3}$ cách chọn a, b, c
      Vậy có $1\times \boldsymbol{A_4^3} = 24$ số.
    - Nếu d = 6 thì có 1 cách chọn d.
      Có 3 cách chọn a (vì a khác 0 và a khác d)
      Khi đó có $\boldsymbol{A_3^2}$ cách chọn b, c.
      Vậy có $1\times 3\times \boldsymbol{A_3^2} = 18$ số
- Vậy có tất cả 24 + 18 = 42 số.

Tổ hợp

Có 6 bao thư khác nhau và 7 con tem khác nhau. Người ta cần chọn ra 3 bao thư và 3 tem thư để dán mỗi bao thư một tem thư. Hỏi có bao nhiêu cách làm như vậy?

Lời giải:

- Chọn 3 bao thư từ 6 bao thư có $\boldsymbol{C_6^3}$ cách
- Chọn 3 tem thư từ 7 tem thư có $\boldsymbol{C_7^3}$ cách
- Dán 3 tem thư đã chọn vào 3 bao thư đã chọn ta có 3! Cách
- Vậy số cách thực hiện theo yêu cầu bài toán là $\boldsymbol{C_6^3} \times \boldsymbol{C_7^3} \times \boldsymbol{3}! = 4200$ cách

3. Sinh tổ hợp bằng Quay lui:

Bài toán: Cho tập ***S*** = *1, 2, 3, ..., **n***. In ra tất cả các tập con có chính xác ***k*** phần tử của ***S***. Hai tập con là hoán vị của nhau chỉ tính là một.

Ý tưởng: lần lượt xây dựng các số trong dãy sao cho số sau lớn hơn số trước đến khi đủ ***k***. Để tránh trùng lặp, ta luôn dựng các tập con **P** là các dãy thỏa mãn ***P[i] > P[i−1]***, hay mọi phần tử đều lớn hơn hẳn phần tử được dựng trước đó. Giả sử ta đã xây dựng được dãy đến vị trí thứ ***i***, và ***P[i]*** là giá trị cuối cùng được thêm vào. Tại vị trí thứ ***i+1***, do có ***P[i + 1] > P[i]***, nên ta chỉ thử các số từ ***P[i+1]*** đến ***n***.

Cài đặt:

```
#include <bits/stdc++.h>
using namespace std;
int n, k;
vector <int> subset;

void out() {
    for (int i : subset) cout << i << " ";
    cout << "\n";
}
void Gen(int pos) {
    int ed = (subset.empty() ? 0 : subset.back());
    for (int i = ed + 1; i <= n; i++) {
        subset.push_back(i);
        if (subset.size() == k) out();
        else Gen(pos + 1);
        subset.pop_back();
    }
}
int main() {
    cin >> n >> k;
    subset.clear();
    Gen(1); return 0;
}
```

III. THUẬT TOÁN LOANG:

1. Khái niệm:

Loang là một kỹ thuật thường được sử dụng trong các bài toán lập trình thi đấu để đánh số các vùng liền kề trong một ma trận. Kỹ thuật này hoạt động dựa trên nguyên tắc loang số từ một điểm khởi đầu đến các điểm lân cận, tương tự như cách thức loang của nước hoặc dầu trên một bề mặt.

2. Cách cài đặt:

Bài toán: Cho một ma trận kích thước ***n*m*** *(1 ⩽ n, m ⩽ 100)* chỉ gồm các số ***0*** hoặc ***1***. Một vùng liên thông là khu vực có các số ***1*** nằm kề cạnh nhau (trên, dưới, trái, phải). Hãy đếm số vùng liên thông trong ma trận.

INPUT	OUTPUT	GIẢI THÍCH
5 5 1 0 0 1 1 1 1 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0	4	1 0 0 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 1 1 1 0 1 0 0

Lời giải *(đọc cùng với code)*:

- Khai báo, nhập dữ liệu theo yêu cầu.
- Khai báo 2 mảng hằng ***dx*** và ***dy*** là các tọa độ khi loang ô kề cạnh (nếu không hiểu bước này, xem lại *bài BANCO trong contest #1 - Chủ đề Vét cạn* tại **đây**).
- Tạo một mảng hai chiều nhị phân (bool) có kích thước ***n*m*** để khi loang kiểm tra xem ô ***(i, j)*** có phải là ô loang được hay không.
- Duyệt từ đầu đến cuối ma trận, khi gặp ô số ***1*** chưa được đánh dấu thì gọi hàm đê quy để loang. (*)
- Trong hàm đệ quy:
  - Đánh dấu ô hiện tại đã được loang ***(FALSE)***.
  - Duyệt qua 2 mảng hằng, ở mỗi ô kề cạnh, kiểm tra xem ô đó có hợp lệ không (có nằm trong ma trận không; có chứa số ***1*** không; đã được đánh dấu chưa).
  - Nếu ô hợp lệ thì ta gọi đệ quy để loang đến ô đó. Ngược lại, ta quay lui về ô trước đó để kiểm tra ô kề cạnh tiếp theo của ô này).
- Mỗi lần thoát khỏi hàm đệ quy và quay về (*) tức là ta đã loang hết một vùng liên thông, khi này chỉ cần đếm như bình thường theo yêu cầu đề bài.

Code - O(n*m) *(có chú thích)*:

```
#include <bits/stdc++.h>
using namespace std;
int dx[4] = {0, 0, 1, -1}, dy[4] = {1, -1, 0, 0};
int n, m, a[101][101], ans = 0;
bool mark[101][101];

void Loang(int x, int y) {
    mark[x][y] = false; // Đánh dấu ô này là false

    /* Mỗi lần Loang tự gọi lại chính nó, tức là vẫn đang ở trong
    vùng liên thông. Như vậy ta đánh dấu để khi sang các ô kề cận
    sẽ không phát hiện lại ô này nữa. Hoặc khi duyệt ở chương
    trình chính sẽ không phát hiện lại ô này vì nó đã thuộc vùng
    liên thông được loang trước đó */

    for(int i=0; i<4; i++) {
        int tmp_x = x + dx[i];
        int tmp_y = y + dy[i];
        if(tmp_x >= 0 && tmp_x < m && tmp_y >= 0 && tmp_y < n) {

            /* Kiểm tra nếu vị trí (x,y)  vẫn trong phạm vi m*n.
            Nếu vị trí này ở rìa bảng thì không xét tiếp được. */

            if(a[tmp_x][tmp_y]==1 && mark[tmp_x][tmp_y] == true)

            /* Kiểm tra nếu vị các vị trí kề cận là 1 và vị trí
            này chưa được xét đến thì gọi lại Loang với vị trí mới
            (là vị trí kề cận có giá trị 1 vừa xét) */

                Loang(tmp_x, tmp_y);
        }
    }
}
```

```
int main() {
    cin >> m >> n;
    for(int i=0; i<m; i++) {
        for(int j=0; j<n; j++) {
            cin >> a[i][j];
            if(a[i][j] == 1) mark[i][j] = true;
            else mark[i][j] = false;
        }
    }
    for(int i=0; i<m; i++) {
        for(int j=0; j<n; j++) {
            if(a[i][j] == 1 && mark[i][j] == true) {
                ans++;
                Loang(i, j);
            }
        }
    }
    cout << ans; return 0;
}
```

IV. BÀI TẬP VẬN DỤNG:

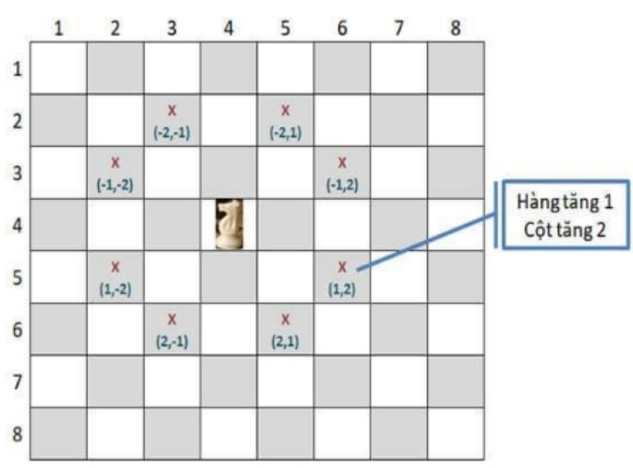
Bài tập 1: Cho tập ***S*** = *1, 2, 3, ..., **n***. In ra tất cả các tập con có chính xác ***k*** phần tử của ***S***. (Giống ví dụ trên nhưng bỏ yếu tố các hoán vị đều tính là một cách)

Bài tập 2: Cho một ma trận kích thước ***n*m*** *(1 ⩽ n, m ⩽ 100)* chỉ gồm các số ***0*** hoặc ***1***. Một vùng liên thông là khu vực có các số ***1*** nằm kề cạnh nhau và 4 hướng chéo. Hãy đếm số vùng liên thông trong ma trận (giống bài ví dụ nhưng bài này loang 8 hướng).

Bài tập 3: Dùng đệ quy tìm ước chung lớn nhất của hai số ***a*** và ***b***.

Bài tập 4:

Cho bàn cờ vua (8x8). Quân mã được đặt ở góc trên cùng bên trái (vị trí (1,1) của bàn cờ). Hãy tìm phương án để quân mã di chuyển qua mỗi ô của bàn cờ đúng một lần.

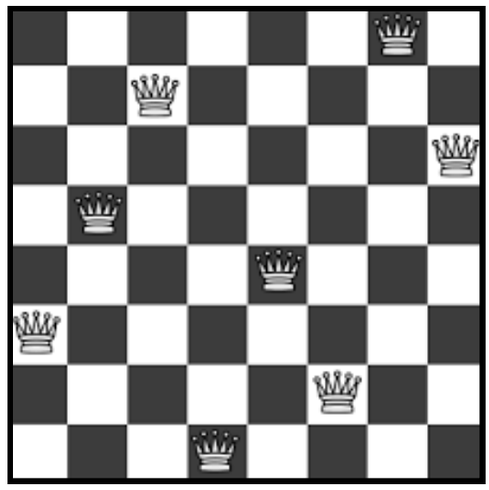


TÁM NƯỚC ĐI CỦA QUÂN MÃ

Bài tập 5: Tìm tất cả các cách xếp ***n*** *(n ≤ 12)* quân hậu lên một bàn cờ ***n×n*** sao cho không có hai quân hậu nào có thể ăn được nhau. Nếu có hai cách là hoán vị của

nhau (về vị trí) thì chỉ tính là một. Hai quân hậu ăn được nhau nếu nằm cùng hàng hoặc cùng cột hoặc cùng một đường chéo.

Hình ảnh minh họa cho trường hợp *n = 8*



V. TỔNG KẾT:

✏️ Đệ quy và quay lui là một trong những kĩ thuật lập trình được xem là "huyền thoại" trong Lập trình thi đấu. Đặc biệt, nó còn có khả năng "hành xác" rất nhiều thế hệ học sinh. Tuy nhiên, khi thông thạo được cách vận dụng thuật toán này, bạn có thể "mở ra một chân trời mới" trong thế giới Lập trình của bản thân và dường như tiến lên một level khác. Do vậy, hãy cố gắng rèn luyện khả năng phân tích và sử dụng đệ quy, quay lui nhé.

✏️ Về sau, đệ quy và quay lui sẽ có đa dạng cách cài đặt và sử dụng hơn. Đặc biệt, giải thuật này có thể được sử dụng xuyên suốt các contest của GG9.

✏️ Đón chờ contest luyện tập sắp tới tại **GDOJ**.

✏️ Tham gia **đoạn chat cộng đồng** để giao lưu cùng các bạn đồng môn và các anh chị học sinh chuyên Tin trên TpHCM.

✏️ Nguồn tham khảo của bài viết: **VNOI**

✏️ Thông tin liên hệ:
- Page: **Căng TIN 2326**
- Group: **Guidelines for Grade 9**
- Gmail: **cangtin2326@gmail.com**

Lúc các bạn đọc được bài viết này thì có lẽ chỉ còn vài chục giờ đếm ngược đến thời khắc giao hòa của đất trời. Chắc hẳn sẽ có người học "Đệ quy & Quay lui" giữa tiếng sóng vỗ rì rập nơi quê hương của muối, sẽ có người học giữa cảnh đồng xanh cò liệng, ai ai cũng đang đắm mình vào cái không khí rạo rực ngày Tết đến Xuân về... Không biết nói gì hơn, chỉ mong rằng các thành viên của đại gia đình GG9 có cái Xuân êm ấm, cái Tết bình yên và một năm mới nhiều thành công nhé ❤️🐧